verisolv: A Unified Artifact Bridging a Numerical ODE/PDE Solver and a Machine-Checked Convergence Proof

verisolv
Open-source research artifact

Abstract

Numerical software for differential equations is trusted on the strength of empirical testing, while the convergence theorems that justify its algorithms live in textbooks, disconnected from the code that runs. We present *verisolv*, a single reproducible monorepo that closes part of this gap: a SciPy-like numerical initial-value and partial-differential-equation solver whose simplest integrator is accompanied by a machine-checked proof of convergence. The library exposes a `solve_ivp` interface with five ODE methods (explicit Euler, classical RK4, adaptive Dormand–Prince RK45, fourth-order Adams–Bashforth, and an implicit backward-differentiation-formula (BDF) stiff solver) and two finite-difference PDE solvers (1-D heat and wave), implemented in Python with an optional Rust kernel exposed through PyO3 that reproduces the fixed-step RK4 trajectory bit-for-bit and the adaptive RK45 trajectory under an identical step-control law. In parallel, a Lean 4 development built on Mathlib formalizes the explicit-Euler recurrence *exactly as implemented* and proves that its global error is $O(h)$ for Lipschitz-continuous right-hand sides under a standard $O(h^2)$ local-truncation bound, with an explicit constant $C = C_2\, T\, e^{LT}$. The proof is complete—it contains no `sorry` and depends only on Lean's three standard axioms—and the build itself runs `#print axioms` to confirm it. The artifact builds and validates end to end with four commands (`pip install`, `pytest`, `cargo test`, `lake build`). We argue that an explicit *contract* tying the verified recurrence to the executed code is what makes "implemented and formally verified" a checkable claim rather than a slogan.

Keywords: formal verification, Lean 4, Mathlib, numerical analysis, ordinary differential equations, Euler method, convergence, Runge–Kutta, PyO3, reproducibility

1 Introduction

The numerical solution of differential equations is among the most widely deployed pieces of scientific software, underpinning simulation across physics, engineering, biology, and finance. The libraries that perform it—SciPy's `integrate` module, SUNDIALS, the MATLAB ODE suite—are validated almost entirely by testing: a method is run on problems with known solutions, and its observed error is checked against the expected rate. This is necessary but not sufficient. Testing demonstrates that an implementation *behaves* on the sampled inputs; it does not establish that the underlying *algorithm* must converge, nor that the code faithfully realizes the algorithm whose convergence the textbook proves.

Formal verification offers the missing guarantee, and interactive theorem provers have matured to the point where the relevant analysis is in reach. The Lean 4 proof assistant [2] and its mathematical library Mathlib [7] now contain enough real analysis—limits, derivatives, Grönwall-type inequalities, the exponential function—to state and prove the convergence of basic numerical schemes. Yet a proof in a vacuum is of limited engineering value: if the verified recurrence is not demonstrably the one the production code executes, the proof certifies a different artifact than the one users run.

This paper presents **verisolv**, an artifact designed around exactly that tension. It is a single monorepo containing three cooperating pillars:

1. a **Python solver library** with a SciPy-like API, five ODE integrators, and two PDE solvers, all deterministic and stability-aware;
2. an optional **Rust numerical kernel** exposed through PyO3 [8] and built with maturin, accelerating the Runge–Kutta inner loops while reproducing the fixed-step RK4 trajectory bit-for-bit; and
3. a **Lean 4 formalization** that proves the explicit Euler method converges with global error $O(h)$ for Lipschitz right-hand sides.

The contribution is not a new integrator or a new theorem—both the Dormand–Prince pair [3] and the convergence of Euler's method are classical. The contribution is the *bridge*: a deliberate, auditable correspondence between the recurrence proved correct in Lean and the recurrence executed in Python, mediated by an explicit interface contract. The explicit Euler step is kept intentionally trivial—no fused stages, no adaptivity—precisely so that the Python loop and the Lean definition

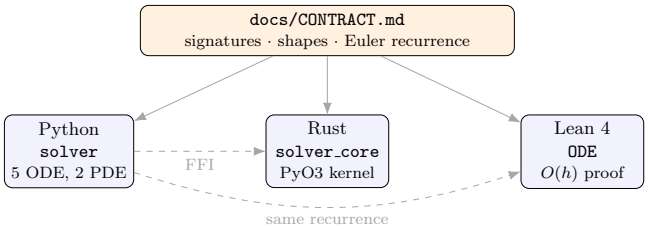

Figure 1. The three pillars and the contract that binds them. The contract pins the interfaces and the Euler recurrence; the Python reference, the Rust kernel, and the Lean proof each conform to it independently.

are visibly the same object, and neither can drift from the other without violating the shared contract.

We make the following claims, each backed by an automated check:

- **The library is real and correct on its tests.** A `pytest` suite of 23 tests checks per-method accuracy, order of convergence, energy conservation, stiff stability, and agreement with SciPy (§7).
- **The Rust path is a transparent accelerator.** Compiled RK4 reproduces the Python trajectory to floating-point identity, and the extension is strictly optional: absent it, the library runs unchanged (§4).
- **The Euler convergence proof is complete.** It contains no `sorry`, and `#print axioms` reports dependence only on `propext`, `Classical.choice`, and `Quot.sound`—Lean's three standard axioms, with the proof-hole axiom `sorryAx` absent (§5).
- **The whole artifact is reproducible.** Four commands build and validate every pillar from a clean checkout (§7).

2 System Architecture

2.1 Monorepo layout

verisolv is organized as a monorepo with one directory per concern: `python_solver/` (the importable `solver` package and its tests), `rust_core/` (the `solver_core` crate and its maturin packaging), `lean/` (the Lean 4 project and its `ODE` library), `benchmarks/` (the SciPy comparison harness), and `docs/` (the interface contract and mathematical notes). Keeping the three language ecosystems in one repository is what lets the artifact tell a single coherent story while still allowing each pillar to be built and tested in isolation: `pytest` runs with no Rust present, `cargo test` runs with no Python interpreter, and `lake build` needs neither.

2.2 The interface contract

A plain-text *contract* (`docs/CONTRACT.md`) is the source of truth for module boundaries, function signatures, return shapes, and—critically—the exact form of the Euler recurrence. Because three independent implementations (Python, Rust, Lean) must agree, the contract pins details that would otherwise drift: every ODE stepper is a free function returning a canonical (t, y, info) triple with the state array in SciPy $(n_{\text{states}}, n_{\text{times}})$ layout; the adaptive step-control law is specified constant-for-constant so the Python and Rust RK45 produce identical trajectories; and the Euler recurrence is fixed as $y_{k+1} = y_k + h\,f(t_k, y_k)$ with the requirement that the Lean development match it. The contract is an engineering artifact, not user documentation, and it is the mechanism by which "the same recurrence" is more than a figure of speech.

2.3 Dispatch and the failure boundary

The user-facing driver `solve_ivp` is a thin dispatcher: it resolves the method name, validates the time span and initial state, wraps the user's right-hand side f in an evaluation counter at the driver boundary (so every method reports a consistent `nfev` regardless of internal bookkeeping), calls the selected stepper, and packages the result. Stepper exceptions are contained and surfaced as a result object with `success=False` rather than propagating, so the public API always returns a well-formed value.

3 The Numerical Solver Library

3.1 ODE integrators

The library provides five integrators spanning the standard taxonomy.

Explicit Euler. is the first-order scheme $y_{k+1} = y_k + h\,f(t_k, y_k)$. It is intentionally the simplest stepper and serves as the *verified anchor*: its loop is line-for-line the recurrence formalized in Lean (§5).

Classical RK4. is the fourth-order explicit Runge–Kutta method with the standard $(1, 2, 2, 1)/6$ weighting. Its global error is $O(h^4)$; halving the step reduces the error by a factor near 16, which the test suite checks directly.

Dormand–Prince RK45. is an embedded 5(4) pair [3]. Six stages (seven under the first-same-as-last optimization) yield both a fifth-order solution that advances the state and a fourth-order companion whose difference estimates the local error. The step is accepted when the root-mean-square (RMS) scaled error norm satisfies $\mathrm{err} \le 1$, and the next step is

$$h_{\text{new}} = h \cdot \mathrm{clip}\Big(0.9\,\mathrm{err}^{-1/5},\ 0.2,\ 5.0\Big)\,,$$

with rejected steps retried at a smaller h. The integrator lands exactly on the final time and reuses the last stage of an accepted step as the first stage of the next.

Adams–Bashforth 4. is an explicit linear multistep method, $y_{k+1} = y_k + \frac{h}{24}(55f_k - 59f_{k-1} + 37f_{k-2} - 9f_{k-3})$, bootstrapped with RK4 for the first three steps. It is fourth-order but uses only one new right-hand-side evaluation per step.

BDF1/BDF2. is a simplified but genuine implicit stiff solver. It bootstraps with backward Euler (BDF1) and continues with BDF2, solving the implicit system at each step by Newton's method with a forward-difference Jacobian and an LU solve. Backward Euler is A-stable (indeed L-stable), so the method stays bounded on stiff problems where explicit schemes would require prohibitively small steps. Concretely, each step solves the nonlinear residual $G(z) = z - y_k - h\,f(t_{k+1}, z) = 0$ (for BDF1) by the Newton update $z \leftarrow z - J^{-1}G(z)$ with $J = I - h\,\partial f/\partial y$ approximated column-by-column by forward differences. A subtle but real bug surfaced during testing: seeding Newton with the explicit-Euler predictor $y_k + hf$ diverges on stiff problems, where $|h\,\partial f/\partial y| \gg 1$ flips the predictor's sign and the absolute residual at that bad guess can fall under tolerance before Newton corrects it. Warm-starting from the previous accepted state—sign-correct and standard for implicit integrators—fixes it, a reminder that the "simplified" label hides genuine numerical care.

3.2 PDE solvers

Two 1-D finite-difference solvers follow the same "real, deterministic, stability-aware" philosophy, each emitting a Courant–Friedrichs–Lewy (CFL) warning rather than silently producing garbage when an explicit scheme's stability criterion is violated.

The **heat equation** $u_t = \alpha u_{xx}$ offers an explicit forward-time centered-space (FTCS) scheme, conditionally stable for $r = \alpha\,\Delta t/\Delta x^2 \le \frac{1}{2}$, and an implicit Crank–Nicolson scheme that is unconditionally stable and second-order in both variables, solving a tridiagonal system each step with a sparse direct solver. The **wave equation** $u_{tt} = c^2 u_{xx}$ uses explicit leapfrog, stable under the Courant condition $C = c\,\Delta t/\Delta x \le 1$, with a second-order Taylor start that injects the initial velocity.

3.3 Stability analysis

The stability criteria above are not folklore constants but the output of von Neumann analysis, which the solvers encode as runtime warnings. Substituting a Fourier mode $u_j^k = G^k e^{\mathrm{i}\beta x_j}$ into the FTCS update gives the amplification factor

$$G(\beta) = 1 - 4r\sin^2\!\left(\tfrac{\beta\Delta x}{2}\right),$$

and $|G| \le 1$ for all β forces $r \le \frac{1}{2}$. The same substitution into Crank–Nicolson yields

$$G(\beta) = \frac{1 - 2r\sin^2(\beta\Delta x/2)}{1 + 2r\sin^2(\beta\Delta x/2)},$$

whose magnitude is at most 1 for *every* $r > 0$, hence unconditional stability. For leapfrog the amplification factor satisfies a quadratic whose roots lie on the unit circle exactly when $C \le 1$—the discrete shadow of the Courant–Friedrichs–Lewy principle that the numerical domain of dependence must contain the physical one. Each solver computes its mesh ratio, compares it to the threshold, and emits a typed `CFLWarning` when the explicit criterion is violated, so an unstable configuration is reported rather than silently returning a diverging field. We validate the implicit and explicit schemes against the analytic fundamental mode $u(x,t) = \sin(\pi x/L)\,e^{-\alpha(\pi/L)^2 t}$ for heat and the standing wave $u(x,t) = \sin(\pi x/L)\cos(\pi c t/L)$ for the wave equation.

4 The Rust Kernel and the Foreign-Function-Interface Boundary

The performance-critical inner loops of RK4 and RK45 are implemented in Rust in the `solver_core` crate, compiled to a PyO3 extension module and packaged with maturin as an `abi3` (stable application-binary-interface) wheel—one binary for CPython 3.10+. The design goal is a *safe* and *testable* foreign-function-interface (FFI) boundary, achieved by a clean split.

The numerically heavy cores—`rk4_core`, `rk45_core`, `bdf1_core`, and a tridiagonal Thomas solver—take the right-hand side as a native Rust closure `&mut dyn FnMut (f64, &[f64])-> Vec<f64>` and contain *no* PyO3 types. Consequently the crate's unit tests pass a native Rust closure and run under plain `cargo test` with no Python interpreter; 20 such tests cover accuracy, adaptivity, determinism, stiff stability, and the linear solver.

The only Python-aware code is the thin `#[pyfunction]` layer, which extracts the initial state, builds the Rust closure from the user's Python callable (invoked under the global interpreter lock, GIL), and records the first error or shape mismatch in an `Option<PyErr>`. On error the closure returns a zero derivative rather than a NaN, guaranteeing the adaptive loop terminates promptly; the wrapper then re-raises the recorded error. The Rust RK45 uses the same step-control law as the Python version, so an accelerated run reproduces the reference trajectory. For fixed-step RK4 the agreement is exact: our test suite asserts that the compiled and pure-Python trajectories are identical to within 10^{-12}, and in practice the maximum observed difference is 0.

On the Python side, the bridge imports `solver_core` inside a `try/except` that can never raise. Success

sets `RUST_AVAILABLE = True` and re-exports the entry points; any failure (missing wheel, ABI mismatch) sets it to `False`. The driver consults this flag when `use_rust=True`: if the extension is absent it warns once and falls back to pure Python. The accelerator is thus entirely optional.

5 Formal Verification of Euler Convergence

The Lean development (`lean/ODE/EulerConvergence.lean`) proves that the explicit Euler method converges with global error $O(h)$. It is built against Mathlib pinned to a fixed release, with the toolchain pinned in `lean-toolchain` for reproducibility.

5.1 Setup

We solve $y'(t) = f(t, y)$ with $y_k \approx y(t_k)$ on the uniform grid $t_k = t_0 + kh$. All data and hypotheses are bundled in a structure:

```
structure EulerData where
  f : ℝ→ℝ→ℝ
  L : ℝ
  hL : 0 ≤L
  lipschitz : ∀t u v : ℝ, |f t u - f t v| ≤L * |u - v|
  h : ℝ
  hh : 0 < h
  t0 : ℝ
  y0 : ℝ
  Y : ℕ→ℝ-- the Euler iterates
  hY0 : Y 0 = y0
  y : ℝ→ℝ-- the exact solution
  hinit : y t0 = y0
  C₂ : ℝ
  hC₂ : 0 ≤C₂
  hYrec : ∀k, Y (k+1) = Y k + h * f (t0 + k*h) (Y k)
  trunc : ∀k, |y (t0+(k+1)*h)
      - (y (t0+k*h) + h * f (t0+k*h) (y (t0+k*h)))| ≤C₂
          ↪  * h^2
```

The field `hYrec` is the Euler recurrence, identical to the Python loop in `methods/euler.py`. We take two analytic facts as *hypotheses*: that f is Lipschitz in its state argument (`lipschitz`, constant L), and that the exact solution's one-step truncation residual is bounded by $C_2 h^2$ (`trunc`). The latter is the standard consequence of a bounded second derivative ($C_2 = M/2$ with $M = \sup |y''|$); we assume it directly rather than re-deriving Taylor's theorem, which keeps the development self-contained while the convergence argument is proved in full.

5.2 The error recurrence

Define the global error $e_k = |Y_k - y(t_k)|$. Subtracting the Euler step from the exact-solution Taylor expansion and applying the triangle inequality and Lipschitz continuity yields the one-step recurrence, proved completely:

```
theorem error_recurrence (k : ℕ) :
    D.e (k+1) ≤(1 + D.h * D.L) * D.e k + D.C₂ * D.h^2
```

The proof writes the next-step error as a sum of three contributions,

$$Y_{k+1} - y(t_{k+1}) = \underbrace{\big(Y_k - y(t_k)\big)}_{A} + \underbrace{h\big(f(t_k, Y_k) - f(t_k, y(t_k))\big)}_{B} + \underbrace{C_k}_{\text{trunc.}},$$

where C_k is the (negated) local truncation residual. The term $|A|$ is exactly e_k; for $|B|$ we pull out the positive step and apply the Lipschitz hypothesis, giving $|B| \le hL\, e_k$; and $|C_k| \le C_2 h^2$ is the `trunc` field. Mechanizing this is more delicate than the paper mathematics suggests: Lean's `set` tactic folds the abbreviations A, B, C_k, after which a naïve `rw` no longer finds the unfolded pattern, so the bounds must be stated against the folded names and combined with `linarith`. The triangle inequality itself is applied twice through Mathlib's `abs_add_le`.

5.3 Discrete Grönwall

The heart of the argument is a self-contained discrete Grönwall inequality:

```
theorem discrete_gronwall {a b : ℝ} {e : ℕ→ℝ}
    (ha : 1 ≤a) (hb : 0 ≤b) (he0 : e 0 ≤0)
    (hrec : ∀k, e (k+1) ≤a * e k + b) :
    ∀n, e n ≤b * n * a ^ n
```

It is proved by induction on n. We deliberately use the polynomial-times-exponential form $e_n \le b\, n\, a^n$ rather than the geometric quotient $b(a^n - 1)/(a - 1)$: the former needs no division and no $a \neq 1$ side condition, yet is exactly strong enough for the $O(h)$ result. The base case is $e_0 \le 0 = b \cdot 0 \cdot a^0$. For the step, from $e_{k+1} \le a\, e_k + b$ and the inductive hypothesis $e_k \le b\, k\, a^k$ we obtain $a\, e_k \le a \cdot b\, k\, a^k = b\, k\, a^{k+1}$, and since $a \ge 1$ implies $a^{k+1} \ge 1$ and $b \ge 0$ we have $b \le b\, a^{k+1}$; adding gives $e_{k+1} \le b\, k\, a^{k+1} + b\, a^{k+1} = b\,(k+1)\, a^{k+1}$. The lone subtlety in Lean is that the natural-number cast of $k+1$ must be normalized with `push_cast` before `ring` closes the arithmetic, since $\uparrow(k+1)$ and $\uparrow k + 1$ are equal but not definitionally so. We also prove $1 \le a^m$ inline rather than depend on a particular library lemma name, a small hedge against version drift (§7).

5.4 The convergence theorem

Combining the two lemmas with $a = 1 + hL$, $b = C_2 h^2$, the bound $(1 + hL)^n \le e^{nhL} \le e^{LT}$ for $nh \le T$, and $b\, n \le C_2 T h$ gives the main result:

```
theorem euler_global_error_le (T : ℝ) (hT : 0 ≤T) :
    ∃C : ℝ, 0 ≤C ∧
      ∀n : ℕ, (n : ℝ) * D.h ≤T →
        |D.Y n - D.y (D.t n)| ≤C * D.h
```

The witness is the *explicit* constant $C = C_2\, T\, e^{LT}$. Because C is independent of h, halving the step halves

Table 1. The correspondence between executed code and proof.

Lean (`EulerConvergence.lean`)	Python (`euler.py`)
`Y`, `hYrec`	the `y += h*f(t,y)` loop
`t k = t0 + k*h`	the uniform time grid
`L`, `lipschitz`	well-posedness of f
`euler_global_error_le`	first-order rate the tests observe

the guaranteed error bound: the method is first-order convergent.

5.5 Trusting the proof

A proof is only as trustworthy as its dependencies. We verify two things. First, the source contains no `sorry` or `admit`. Second, and more strongly, `#print axioms` on the main theorem reports

```
[propext, Classical.choice, Quot.sound],
```

the three standard axioms underlying Mathlib. The proof-hole axiom `sorryAx` is *absent*, which is a machine-checked guarantee that no gap was tactically hidden anywhere in the dependency graph. The same holds for `discrete_gronwall` and `error_recurrence`.

6 The Bridge

The project's central idea is that the *same* recurrence is implemented and verified. The correspondence is concrete and maintained on three levels.

First, the recurrences are textually identical (Table 1); the Python module and the Lean file cross-reference each other by path. The Python inner loop reads

```python
# Lean recurrence: y_{k+1} = y_k + h f(t_k, y_k)
for k in range(n_steps):
    yk = yk + h_eff * f(tk, yk)
    tk = t0 + (k + 1) * h_eff
```

and the Lean field `hYrec` states `Y (k+1)= Y k + h * f (t0 + k*h)(Y k)`—the same object, modulo the grid-time abbreviation $t_k = t_0 + kh$ that both sides share.

Second, the contract (§2) pins the recurrence, so neither side can change without an auditable edit to the shared specification. Third, the empirical and the formal meet in the evaluation: the test suite *observes* first-order accuracy on $y' = -y$ (with f Lipschitz, $L = 1$), the exact rate the theorem *guarantees*. We are explicit about scope: only Euler is formally verified. The higher-order methods are validated numerically. Euler is the verified anchor; the rest is empirical evidence built outward from it.

Table 2. Maximum error, wall-clock time, and right-hand-side evaluations versus SciPy. "ours" denotes the verisolv solver; time is the minimum over repeated runs. Lower is better throughout.

Problem	Method	Max error	Time (s)	`nfev`
exp. decay	RK45 (ours)	8.70×10^{-8}	0.010	248
exp. decay	RK45 (scipy)	9.93×10^{-8}	0.005	248
harmonic	RK45 (ours)	2.49×10^{-6}	0.025	728
harmonic	RK45 (scipy)	2.50×10^{-6}	0.018	716
Lotka–Volterra	RK45 (ours)	4.02×10^{-4}	0.054	1400
Lotka–Volterra	RK45 (scipy)	3.89×10^{-4}	0.045	1394
Robertson (stiff)	BDF (ours)	1.65×10^{-6}	1.56	20058
Robertson (stiff)	BDF (scipy)	6.87×10^{-7}	0.066	329

7 Evaluation

7.1 Reproducibility

From a clean checkout the artifact builds and validates with four commands: `pip install -e .` (the Python package), `pytest` (the test suite), `cargo test` (the Rust kernels), and `lake exe cache get && lake build` (the proof). An optional fifth step, `maturin develop --release` followed by a second `pytest` run, builds the Rust extension and activates the accelerated-path cross-checks. All toolchain versions are pinned. The test suite reports 23 passing tests; with the Rust extension built, the previously skipped cross-checks activate and confirm Rust/Python parity. The Rust crate reports 20 passing unit tests. The Lean library builds cleanly, and `#print axioms`—run as part of `lake build`—confirms the proof depends only on the three standard axioms with `sorryAx` absent.

7.2 Numerical accuracy versus SciPy

Table 2 reports our solvers against `scipy.integrate.solve_ivp` on four problems, measuring maximum error against an analytic or high-accuracy reference. On exponential decay our adaptive RK45 reaches 8.7×10^{-8} at 248 right-hand-side evaluations, matching SciPy's 9.9×10^{-8} at the same evaluation count—unsurprising, as both implement the same Dormand–Prince pair. On the harmonic oscillator and Lotka–Volterra the agreement is similarly close. On the stiff Robertson kinetics our fixed-step BDF reaches comparable accuracy to SciPy's adaptive variable-order BDF, though at higher cost—an expected consequence of fixed stepping, and a natural target for future adaptivity.

7.3 The verified rate, observed

The test suite checks that explicit Euler on $y' = -y$ exhibits genuine first-order behavior—accurate to a loose tolerance but not accidentally exact—and that RK4 exhibits its fourth-order rate, with the error ratio between

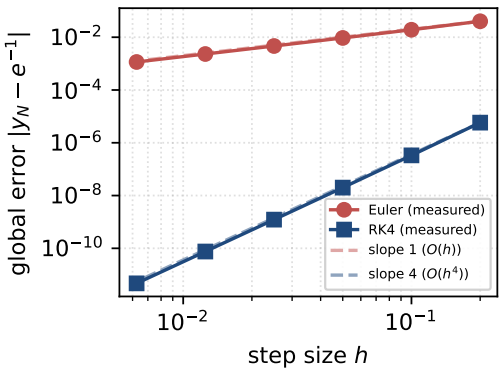

Figure 2. Global error versus step size for explicit Euler and RK4 on $y' = -y$, log–log axes, with reference slopes. A least-squares fit gives empirical orders 1.02 (Euler) and 4.04 (RK4). The measured Euler slope of 1 is the numerical realization of the $O(h)$ rate proved by `euler_global_error_le`.

step sizes $h = 0.1$ and $h = 0.05$ falling in $[12, 20]$ (the theoretical value is 16). Figure 2 makes the rates visible across a sweep of step sizes: a least-squares fit on the log–log data recovers empirical orders of 1.02 for Euler and 4.04 for RK4. The first-order Euler observation is the numerical shadow of `euler_global_error_le`—the theorem and the measurement agree.

7.4 Test-suite coverage

The 23 `pytest` cases are not smoke tests but targeted checks of numerical properties. For ODEs they verify: per-method accuracy on exponential decay (Euler loose but genuinely first order, RK4/RK45 to 10^{-6}); the harmonic oscillator against cos/− sin with energy conservation over a full period; Lotka–Volterra positivity and oscillation together with an endpoint cross-check of our RK45 against SciPy's to 10^{-4}; bitwise determinism of repeated calls; the RK4 order-of-convergence ratio; AB4 and BDF accuracy on decay; and BDF stability on a stiff linear problem. When the Rust extension is present, two further cases assert Rust/Python RK4 identity and a version probe. For PDEs the suite checks Crank–Nicolson against the analytic heat mode, the FTCS CFL warning firing exactly when $r > \frac{1}{2}$, the FTCS stable regime, the leapfrog standing wave, and the wave Courant warning. The 20 Rust unit tests independently cover the kernels, the adaptive controller, the Thomas solver, and the stiff path.

7.5 Reproducibility as an engineering discipline

Pinning the proof to a fixed Mathlib release is not a formality. Mathlib evolves rapidly, and lemma names drift between versions: in bringing the development up on the pinned toolchain we encountered exactly this, with the triangle inequality formerly spelled `abs_add` now requiring `abs_add_le` and the power-monotonicity lemma `pow_le_pow_left` renamed to `pow_le_pow_left₀`. A proof script that built last year may simply fail to elaborate today. We therefore pin both the Lean toolchain (`lean-toolchain`) and the exact Mathlib revision, and the build step uses `lake exe cache get` to fetch Mathlib's prebuilt object files—without it, a from-source Mathlib compilation dominates build time by orders of magnitude. A second lesson concerns robustness: where possible we replaced automation that silently depends on the ambient hypothesis set (for instance `positivity` ↪ , which could not see a structure field's nonnegativity proof) with explicit lemma applications such as `mul_nonneg`, trading brevity for stability against library evolution. These choices are why the artifact's four-command validation remains reproducible rather than aspirational.

8 Related Work

Verified numerics. Formal verification of numerical analysis has a substantial history in higher-order logic. Immler and Hölzl verified ODE flows and a Runge–Kutta method in Isabelle/HOL, including rigorous enclosures of solutions [5]; Immler's later work produced a *verified, executable* ODE solver with rigorous Runge–Kutta integration, obtained by Isabelle code extraction and applied to the Lorenz attractor [4]. That line of work is the closest prior art to our pitch: it achieves a verified-to-executable path via extraction with rigorous enclosures, whereas verisolv deliberately keeps the verified core minimal and bridges it to independently written multi-language code by an explicit contract—a lighter, more replicable correspondence at the cost of the extraction guarantee. Boldo and collaborators verified a C program implementing a finite-difference scheme for the wave equation, bounding both the method and round-off error [1]. The CompCert verified C compiler [6], with its Flocq-based IEEE-754 semantics, shows that the compilation and floating-point layers can themselves be verified—complementary to the algorithm-level guarantee we provide, and a route toward closing the real-versus-float gap we leave open (§9). Our aim is narrower and more pedagogical: a tight, finishable Euler convergence proof in Lean 4/Mathlib, deliberately wired to a running multi-language library.

Lean and Mathlib. Lean 4 [2] and Mathlib [7] provide the real analysis our proof rests on. Mathlib already contains Grönwall-type inequalities and the Picard–Lindelöf theorem for ODE existence and uniqueness; we use the elementary parts and prove a self-contained discrete Grönwall bound tailored to the Euler recurrence. We are clear-eyed about the formalization's standing:

its novelty is not new analysis—the discrete Grönwall induction and error recurrence are elementary by Mathlib's standards—but the *code-tied packaging*, the fact that the verified recurrence is the one a running solver executes. For a formalization-focused reading, the natural way to deepen the Lean content is to discharge the local-truncation hypothesis from a bound on y'' using Mathlib's Taylor-theorem infrastructure, which we leave to future work (§9).

Numerical libraries. SciPy's `solve_ivp` [9] is the reference API our design echoes. The Rust acceleration via PyO3 [8] follows the now-common pattern of a Python front-end over a compiled kernel.

9 Discussion and Future Work

verisolv is deliberately scoped: the verified core is Euler, and the truncation bound is a hypothesis rather than a theorem derived from a bound on y''. The clearest next step is to discharge that hypothesis inside Lean using Mathlib's Taylor-theorem infrastructure, removing the last analytic assumption. Beyond that, the same contract-driven methodology extends to verifying the order of RK4 or the A-stability of backward Euler, and to connecting the discrete PDE schemes to their continuous stability analyses. A more ambitious direction is to narrow the implementation gap further—for instance by extracting executable Lean or by generating the Python and Rust Euler loops from the verified definition, so that the correspondence is mechanical rather than maintained by contract.

Threats to validity. We are candid about what the artifact does and does not establish. The proof certifies the *algorithm*—the abstract Euler recurrence over the reals—not the floating-point Python code that approximates it; rounding error is outside the model, as is the equivalence between the real-number recurrence and its IEEE-754 realization. The correspondence between the Lean recurrence and the executed loop is maintained *socially*, by the contract and by cross-referencing comments, not mechanically; a sufficiently careless edit could desynchronize them without tripping a build failure, which is precisely why we regard mechanical generation as the most valuable future direction. The Lipschitz and truncation facts are assumed rather than derived. Finally, the higher-order and PDE methods carry no formal guarantee at all—their evidence is empirical. None of these caveats undercut the central claim, which is narrow by design: that a specific, classical convergence result can be proved with no holes and tied, auditably, to running code.

10 Conclusion

We presented verisolv, a single reproducible artifact in which a SciPy-like numerical solver and a machine-checked convergence proof share one narrative. The library provides five ODE integrators and two PDE solvers with an optional Rust kernel that reproduces the fixed-step RK4 trajectory exactly; the Lean development proves explicit Euler converges at $O(h)$ with an explicit constant and no proof holes, certified by `#print axioms`. The connective tissue is an explicit contract that keeps the verified recurrence and the executed code provably in step. We believe this contract-driven bridge—rather than either pillar alone—is the reusable idea: it turns "implemented and formally verified" from a claim into something a reader can check with four commands.

Artifact Availability

The complete artifact—Python package, Rust crate, Lean proof, benchmarks, and documentation—builds and validates with the four commands of §7. Toolchain versions are pinned for reproducibility.

References

[1] Sylvie Boldo, François Clément, Jean-Christophe Filliâtre, Micaela Mayero, Guillaume Melquiond, and Pierre Weis. 2013. Wave Equation Numerical Resolution: A Comprehensive Mechanized Proof of a C Program. *Journal of Automated Reasoning* 50, 4 (2013), 423–456. doi:10.1007/s10817-012-9255-4

[2] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28 (Lecture Notes in Computer Science, Vol. 12699)*. Springer, 625–635. doi:10.1007/978-3-030-79876-5_37

[3] J. R. Dormand and P. J. Prince. 1980. A Family of Embedded Runge–Kutta Formulae. *J. Comput. Appl. Math.* 6, 1 (1980), 19–26. doi:10.1016/0771-050X(80)90013-3

[4] Fabian Immler. 2018. A Verified ODE Solver and the Lorenz Attractor. *Journal of Automated Reasoning* 61, 1–4 (2018), 73–111. doi:10.1007/s10817-017-9448-y

[5] Fabian Immler and Johannes Hölzl. 2012. Numerical Analysis of Ordinary Differential Equations in Isabelle/HOL. In *Interactive Theorem Proving (ITP 2012) (Lecture Notes in Computer Science, Vol. 7406)*. Springer, 377–392. doi:10.1007/978-3-642-32347-8_26

[6] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* 52, 7 (2009), 107–115. doi:10.1145/1538788.1538814

[7] The mathlib Community. 2020. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. ACM, 367–381. doi:10.1145/3372885.3373824

[8] The PyO3 Project. 2024. PyO3: Rust Bindings for the Python Interpreter. https://github.com/PyO3/pyo3. Accessed 2026.

[9] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17 (2020), 261–272. doi:10.1038/s41592-019-0686-2

A The Discrete Grönwall Proof in Full

For concreteness we reproduce the complete, `sorry`-free Lean source of the discrete Grönwall lemma (§5), the inductive heart of the convergence argument. It depends only on Mathlib's three standard axioms.

```
theorem discrete_gronwall {a b : ℝ} {e : ℕ→ℝ}
    (ha : 1 ≤a) (hb : 0 ≤b) (he0 : e 0 ≤0)
    (hrec : ∀k, e (k + 1) ≤a * e k + b) :
    ∀n, e n ≤b * n * a ^ n := by
  have ha0 : 0 ≤a := le_trans zero_le_one ha
  -- 1 ≤a^n, proved inline to avoid a library name.
  have hpow_ge_one : ∀m : ℕ, (1 : ℝ) ≤a ^ m := by
    intro m
    induction m with
    | zero => simp
    | succ k ih => rw [pow_succ]; nlinarith [ih, ha]
  intro n
  induction n with
  | zero => simpa using he0
  | succ k ih =>
    push_cast
    have hstep : e (k + 1) ≤a * e k + b := hrec k
    have hmul : a * e k ≤a * (b * k * a ^ k) :=
      mul_le_mul_of_nonneg_left ih ha0
    have hb_le : b ≤b * a ^ (k + 1) := by
      nlinarith [hpow_ge_one (k + 1), hb]
    calc
      e (k + 1) ≤a * e k + b := hstep
      _ ≤a * (b * k * a ^ k) + b := by linarith
      _ = b * k * a ^ (k + 1) + b := by rw [pow_succ];
          ↪ ring
      _ ≤b * k * a ^ (k + 1)
            + b * a ^ (k + 1) := by linarith
      _ = b * ((k : ℝ) + 1) * a ^ (k + 1) := by ring
```

The convergence theorem instantiates this with $a = 1+hL$ and $b = C_2h^2$, bounds $(1+hL)^n \le e^{LT}$ via Mathlib's `Real.add_one_le_exp` together with `pow_le_pow_left`$_0$, and discharges the final nonnegativity obligations with explicit `mul_nonneg` applications rather than `positivity`, for the robustness reasons discussed in §7.